

Introducción a Python

- Introducción
- Qué es Python
 - + Versiones de Python
- Cómo trabajar con Python
- Algunos conceptos básicos
- Funciones integradas
- Funciones de entrada y salida
- Módulos en Python
- Estructuras de control
 - + Instrucciones alternativas: if, else, elif
 - + Instrucciones repetitivas: while, for
 - + Otras instrucciones: break, continue, pass
- Definición de funciones
- Tipos de datos complejos
 - + Tipo de datos LISTA
 - + Tipo de datos DICCIONARIO
- Excepciones
- Fuentes de información

INTRODUCCIÓN

Si estás leyendo este documento es porque ya tienes nociones básicas de programación y has decidido aprender Python. En cada unidad didáctica de Acceso a Datos iremos ampliando sobre Python.

En esta unidad didáctica vamos a hacer un repaso general de los conceptos básicos necesarios para trabajar con Python.

¡Empezamos!

QUÉ ES PYTHON

Python es un lenguaje de programación que fue creado a finales de los ochenta por Guido van Rossum en el Centro para las Matemáticas y la Informática (CWI, Centrum Wiskunde & Informatica) de los Países Bajos.

Guido es un gran admirador del grupo humorístico británico Monty Python y decidió bautizar al nuevo lenguaje inspirándose en el nombre de dicho grupo.

Según su página oficial (<https://www.python.org/>), “Python es un lenguaje de programación que te permite trabajar rápido e integrar sistemas de una manera más efectiva”.

VERSIONES DE PYTHON

A día de hoy disponemos de la versión 3.10 de Python tanto para Linux/Unix como para Windows o Mac OS. Este documento está basado en la versión 3.8.5.

En su página oficial (<https://www.python.org/downloads/>) podemos encontrar esta versión e incluso versiones anteriores o versiones específicas para otras plataformas como AIX, IBM i, iOS, iPadOS, Solaris, etc .

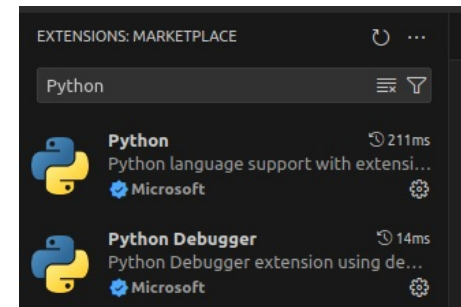
CÓMO TRABAJAR CON PYTHON

Existen multitud de formas para crear y probar programas en Python. Desde utilizar el intérprete de comandos que incorpora la instalación básica del lenguaje hasta usar plataformas on-line donde podemos olvidarnos completamente de cualquier tarea de instalación. Cada opción tiene sus ventajas e inconvenientes, así que la elección de una u otra dependerá de tus necesidades y recursos. No obstante, utilices la opción que utilices, debes saber que todos los programas de Python se almacenan en ficheros con extensión .py

Python se encuentra preinstalado en muchos sistemas operativos como Linux y Mac OS X en una versión suficientemente reciente. En aules se adjunta un documento para instalar en Linux, Windows y Mac OS X si deseamos la versión más reciente.

Una vez instalado Python (nos referimos al intérprete) es muy recomendable instalar el gestor de paquetes pip y virtualenv. Con pip podemos instalar paquetes Python de forma muy sencilla y sin permisos de administrador (pip install nombre_paquete). virtualenv permite el desarrollo de aplicaciones Python en un entorno aislado de las dependencias del sistema, se verá y utilizará en la unidad didáctica de ORM SQLAlchemy.

Un editor muy recomendado es IDE Thonny, un entorno de desarrollo integrado que incorpora un depurador para detectar errores en la prueba de programas. Nosotros vamos a trabajar con VSCode y las extensiones básicas de Python.



ALGUNOS CONCEPTOS BÁSICOS

En Python las variables no hace falta declararlas antes de ser utilizadas ya que es en el momento de ejecución del programa cuando se conoce el tipo de dato que guarda cada variable y los operadores y funciones que se le pueden aplicar. A este mecanismo se le conoce como tipado dinámico de variables.

En Python todos los elementos son objetos. Es decir, una variable de tipo entero será una variable que referenciará a un objeto de tipo entero o una variable de tipo cadena de texto será una variable que referenciará a un objeto de tipo cadena de texto. En la siguiente tabla puedes ver los tipos de datos más usuales en Python:

Clasificación	Clase en Python	Descripción
Números	int	Número entero. Ejemplos: 3, 55, 1948.
	float	Número real. Ejemplos: 10.3348, .98344, 193., 1.9.
	complex	Número complejo con parte real y parte imaginaria. Ejemplos: 3.2+6j, 0.1+4.55j
Booleanos	bool	Presentan dos valores: <i>True</i> y <i>False</i> equivalentes a los valores verdadero o falso de la lógica.

ALGUNOS CONCEPTOS BÁSICOS

Clasificación	Clase en Python	Descripción
Secuencias	str	Cadenas de texto o secuencias de caracteres. Ejemplo: "Buenos días"
	list	Listas de elementos de cualquier tipo que pueden ser modificables. Ejemplos: [1, "ya", True], ['lunes', 'jueves', 'viernes', 'domingo']
	tuple	Las tuplas son listas que no se pueden modificar. Se crean con unos valores y no pueden cambiarse. Ejemplos: (1, "ya", True), ('lunes', 'jueves', 'viernes', 'domingo')
	bytes	Secuencias de enteros que representa códigos ASCII y que no se pueden modificar. Ejemplos: b'Python is interesting.', b'\x00\x00\x00\x00\x00'
	bytearray	Similar a <i>bytes</i> pero sí se pueden modificar.

ALGUNOS CONCEPTOS BÁSICOS

Clasificación	Clase en Python	Descripción
Conjuntos	set	Conjuntos de datos desordenados pero sin repeticiones de elementos. Se pueden modificar. Ejemplos: {3, 5, 1, 7, 4, 10}, {'lunes', 'jueves'}
	frozenset	Similar a <i>set</i> pero no se puede modificar.
Diccionarios	dict	Permiten almacenar valores a los que se asigna una clave única para acceder. Ejemplos: {'one': 1, 'two': 2, 'three': 3}

Realiza actividad_tipos.py

ALGUNOS CONCEPTOS BÁSICOS

Los operadores son símbolos que nos permiten realizar operaciones con objetos de un programa. Dependiendo de la clase a la que pertenezcan dichos objetos, podremos utilizar unos operadores u otros.

Tipos de operadores	Operadores disponibles
Aritméticos	+, -, *, /, // (división entera), % (módulo de la división entera), ** (potencia)
Comparación	==, !=, >, >=, <, <=
Asignación	=, +=, -=, *=, /=, //=, %=, **=
Booleanos o lógicos	and, or, not
Identidad	is, is not

Realiza actividad_operadores_tipos_básicos.py

FUNCIONES INTEGRADAS

El intérprete de Python dispone de funciones integradas o predefinidas que podemos utilizar en cualquiera de nuestros programas y sin necesidad de importar ningún módulo extra.

Built-in Functions				
<code>abs()</code>	<code>delattr()</code>	<code>hash()</code>	<code>memoryview()</code>	<code>set()</code>
<code>all()</code>	<code>dict()</code>	<code>help()</code>	<code>min()</code>	<code>setattr()</code>
<code>any()</code>	<code>dir()</code>	<code>hex()</code>	<code>next()</code>	<code>slice()</code>
<code>ascii()</code>	<code>divmod()</code>	<code>id()</code>	<code>object()</code>	<code>sorted()</code>
<code>bin()</code>	<code>enumerate()</code>	<code>input()</code>	<code>oct()</code>	<code>staticmethod()</code>
<code>bool()</code>	<code>eval()</code>	<code>int()</code>	<code>open()</code>	<code>str()</code>
<code>breakpoint()</code>	<code>exec()</code>	<code>isinstance()</code>	<code>ord()</code>	<code>sum()</code>
<code>bytearray()</code>	<code>filter()</code>	<code>issubclass()</code>	<code>pow()</code>	<code>super()</code>
<code>bytes()</code>	<code>float()</code>	<code>iter()</code>	<code>print()</code>	<code>tuple()</code>
<code>callable()</code>	<code>format()</code>	<code>len()</code>	<code>property()</code>	<code>type()</code>
<code>chr()</code>	<code>frozenset()</code>	<code>list()</code>	<code>range()</code>	<code>vars()</code>
<code>classmethod()</code>	<code>getattr()</code>	<code>locals()</code>	<code>repr()</code>	<code>zip()</code>
<code>compile()</code>	<code>globals()</code>	<code>map()</code>	<code>reversed()</code>	<code>__import__()</code>
<code>complex()</code>	<code>hasattr()</code>	<code>max()</code>	<code>round()</code>	

Figura 1: Font: <https://docs.python.org/3/library/functions.html>

Identifica aquellas que conozcas ahora y al acabar la unidad didáctica.

FUNCIONES DE ENTRADA Y SALIDA

Para solicitar datos por teclado al usuario podemos utilizar la función input(). Cuando se ejecuta esta función, el programa queda esperando que el usuario introduzca una cadena de texto por teclado y pulse la tecla Intro/Enter.

```
In [7]: texto = input("Escribe una frase, por favor: ")
Escribe una frase, por favor: Buenos días.

In [8]: texto
Out[8]: 'Buenos días.'
```

Figura 2: Uso de input() con enunciado

Si necesitamos pedir por teclado un valor distinto de texto deberemos hacer la conversión al tipo de datos correspondiente. Por ejemplo: int() o float().

```
In [14]: entero = int(input("Dame un número entero: ")) ; entero
Dame un número entero: 15
Out[14]: 15

In [15]: real = float(input("Dame un número real: ")) ; real
Dame un número real: 35.298
Out[15]: 35.298
```

Figura 3: Petición de números por teclado

Por su lado, la función print() permite mostrar información por pantalla al usuario. Podemos imprimir cadenas de texto, números, valores lógicos, listas, diccionarios, etc.

FUNCIONES DE ENTRADA Y SALIDA

Existen distintas formas de utilizar esta función `print()`. Aquí se pueden ver algunos ejemplos:

```
In [35]: print("Hola %s." % ("Pepito"))  
Hola Pepito.
```

```
In [36]: print("%s, trae %d tebeos de %.2f€ cada uno." % ("Pepito", 5, 12.5))  
Pepito, trae 5 tebeos de 12.50€ cada uno.
```

Figura 4: Uso básico del operador %

```
In [19]: print("{} trae {} tebeos de {:.2f}€ cada uno.".format("Pepito", 5, 12.5))  
Pepito trae 5 tebeos de 12.50€ cada uno.
```

Figura 5: Uso básico de {}

```
>>> nom = "Pepito"  
>>> num = 5  
>>> preu = 12.5  
>>> print(f'{nom}, trae {num} tebeos a {preu}€ cada uno.')  
Pepito, trae 5 tebeos a 12.5€ cada uno.
```

Figura 6: Uso básico de f-cadenas

Realiza actividad_funciones_ES.py

MÓDULOS EN PYTHON

Cuando queremos crear un programa en Python, tenemos que crear un fichero de texto plano con extensión .py. Cada uno de estos ficheros es considerado un módulo en Python.

Para importar un módulo debemos utilizar la palabra reservada import seguida del nombre del módulo y de la jerarquía de paquetes a la que pertenezca:

```
import nombre_modulo
```

```
import paquete.nombre_modulo
```

```
import paquete.subpaquete.nombre_modulo
```

En algunas ocasiones nos puede interesar importar sólo algunos elementos de un módulo y no el módulo completo. En ese caso, debemos indicar el elemento a importar o una lista de elementos separados por coma, tal y como se muestra a continuación:

```
from nombre_modulo import elemento
```

```
from nombre_modulo import elemento1, elemento2, ..., elementon
```

A partir de entonces, podemos utilizar al nombre del elemento directamente en el programa sin necesidad de anteponer el namespace correspondiente.

```
In [23]: from paquete1.modulo11 import resta  
In [24]: resta(10, 3)  
Out[24]: 7
```

Figura 7: Uso de "from" en un "import"

Realiza aritmetico.py y
actividad_módulos.py

ESTRUCTURAS DE CONTROL

En este apartado repasaremos la sintaxis de las distintas estructuras de control que ofrece Python.

INSTRUCCIONES ALTERNATIVAS: IF, ELSE, ELIF

```
if condición:
    instrucción1.1
    ...
    instrucción1.n
else:
    instrucción2.1
    ...
    instrucción2.m
```

```
26 seguir = input("¿Quieres seguir (S/N)?")
27 if seguir == 'S':
28     print("Has elegido seguir.")
29 else:
30     print("Has elegido no seguir.")
31
32
33 num = int(input("Dame un número (1-10):"))
34 if num < 5:
35     print("Sigue intentándolo.")
36 else:
37     print("¡Enhorabuena!")
```

Figura 8: Ejemplo usos básicos de if-else

INSTRUCCIONES ALTERNATIVAS: IF, ELSE, ELIF

```
if condición1:  
    instrucción1,1  
    ...  
    instrucción1,n  
[elif condición2:  
    instrucción2,1  
    ...  
    instrucción2,m  
...  
elif condiciónp:  
    instrucciónp,1  
    ...  
    instrucciónp,q]  
[else:  
    instrucciónr,1  
    ...  
    instrucciónr,s]
```

```
76 num = int(input("Dame un número (1-10):"))  
77 if num <= 2:  
78     print("Muy deficiente")  
79 elif num >= 3 and num <= 4:  
80     print("Insuficiente")  
81 elif num == 5:  
82     print("Aprobado")  
83 elif num == 6:  
84     print("Bien")  
85 elif num >= 7 and num <= 8:  
86     print("Notable")  
87 elif num >= 9:  
88     print("Sobresaliente")
```

Figura 9: Uso de cláusulas elif

Realiza actividad_if_else.py

INSTRUCCIONES REPETITIVAS: WHILE, FOR

```
while condición:  
    instrucción1.1  
    ...  
    instrucción1.n
```

```
91 opcion = input("¿Quieres seguir? (S/N) ")  
92 while ((opcion.upper() != 'S') and (opcion.upper() != 'N')):  
93     opcion = input("¿Quieres seguir? (S/N) ")  
94 print("Opción correcta: ", opcion)
```

Figura 10: Uso de "while" para validar un dato introducido por teclado

```
for elemento in secuencia:  
    instrucción1.1  
    ...  
    instrucción1.n
```

```
110 palabra = "HOLA"  
111 for i in range(len(palabra)):  
112     print(i, '-->', palabra[i])
```

Figura 11: Iterando sobre una secuencia

Realiza actividad_while_for.py

OTRAS INSTRUCCIONES: BREAK, CONTINUE, PASS

Python dispone de dos sentencias que permiten alterar el funcionamiento normal de un bucle: `break` y `continue`.

Cuando se utiliza la sentencia `break` dentro de un bucle provoca la finalización del mismo sin hacer caso de la condición del `while` o del iterador del `for`. Por su parte, la sentencia `continue` da por finalizada la iteración actual y pasa a la siguiente aunque no provoca el fin del bucle completo.

Por último, la sentencia `pass` es una sentencia vacía, es decir, no hace nada. Suele utilizarse para que el programa no dé error de sintaxis cuando se está estructurando un programa pero aún no se ha implementado el conjunto de instrucciones que son necesarias.

```
122 while True:
123     pass
124
125 for i in range(len(palabra)):
126     pass
```

Figura 12: Ejemplo uso de "pass"

Realiza actividad_rangos.py

DEFINICIÓN DE FUNCIONES

Las funciones se definen con la siguiente sintaxis:

```
def nombre_función(param1, ... , paramn):  
    # instrucciones del cuerpo de la función  
    return (elem1, elem2, ... , elemn) #devuelve una tupla
```

La llamada a una función dependerá del número de parámetros de entrada que tenga y los valores que devuelva.

Ejemplo: La función calculadora recibe dos parámetros de entrada y devuelve cuatro resultados de operaciones.

```
1 def calculadora(a,b):  
2     '''Realiza operaciones básicas.'''  
3     suma = a + b  
4     resta = a - b  
5     producto = a * b  
6     division = a / b  
7  
8     return (suma, resta, producto, division)  
9  
10 a = int(input("Dame el operando 1:"))  
11 b = int(input("Dame el operando b:"))  
12 resultado = calculadora(a, b)  
13  
14 for i in range(len(resultado)):  
15     print("Resultado", i, "-->", resultado[i])
```

Figura 13: Return de varios valores

Realiza actividad_funciones.py

Realiza actividad_funciones2.py

```
Dame el operando 1:10  
Dame el operando b:4  
Resultado 0 --> 14  
Resultado 1 --> 6  
Resultado 2 --> 40  
Resultado 3 --> 2.5
```

Figura 14: Ejecución del programa

TIPOS DE DATOS COMPLEJOS

La diferencia fundamental entre los tipos de datos básicos y los complejos es que los primeros son inmutables mientras que los segundos no lo son. La mutabilidad o inmutabilidad de un objeto hace referencia a la capacidad de poder modificar su contenido o no, una vez que ha sido creado.

TIPO DE DATOS LISTA

Las listas son un tipo de datos secuencia mutable que permite almacenar elementos ordenados de cualquier tipo. Para crear una lista, podemos hacerlo de dos formas:

a) Encerrando entre corchetes una lista de elementos separados por comas.

```
In [8]: numeros = [1, 2, 3, 4, 5]; numeros
Out[8]: [1, 2, 3, 4, 5]

In [9]: vocales = ['a', 'e', 'i', 'o', 'u']; vocales
Out[9]: ['a', 'e', 'i', 'o', 'u']
```

Figura 15: Creación de listas con [y]

b) A partir de un tipo secuencia, como una cadena de texto u otra lista, mediante el uso del constructor list().

```
In [26]: numeros
Out[26]: [1, 2, 3, 4, 5]

In [27]: otro = list(numeros); otro
Out[27]: [1, 2, 3, 4, 5]

In [28]: vocales = list("aeiou"); vocales
Out[28]: ['a', 'e', 'i', 'o', 'u']
```

Figura 16: Uso del constructor list()

TIPO DE DATOS LISTA

En la siguiente tabla se recogen algunas operaciones con listas:

Operaciones	Descripción
Acceso a elementos	Utilizamos el operador []: <pre>In [40]: colores[0] = "negro"</pre> <pre>In [41]: colores</pre> <pre>Out[41]: ['negro', 'verde', 'azul', 'rojo', 'azul']</pre>
Recorrido de listas	Accediendo a los elementos directamente: <pre>In [39]: colores</pre> <pre>Out[39]: ['rojo', 'amarillo', 'azul', 'verde']</pre> <pre>In [40]: for color in colores:</pre> <pre>....: print(color, end=" ")</pre> <pre>....:</pre> <pre>rojo amarillo azul verde</pre> Utilizando un índice: <pre>In [43]: numeros</pre> <pre>Out[43]: [5, 2, 3, 4, 1]</pre> <pre>In [44]: for i in range(len(numeros)):</pre> <pre>....: numeros[i] *= 10</pre> <pre>....:</pre> <pre>In [45]: numeros</pre> <pre>Out[45]: [50, 20, 30, 40, 10]</pre>

TIPO DE DATOS LISTA

Operaciones	Descripción
Longitud	Utilizaremos la función len() : <code>In [5]: numeros = [5, 2, 3, 4, 1]</code> <code>In [6]: len(numeros)</code> <code>Out[6]: 5</code> <code>In [7]: colores = ["rojo", "amarillo", "azul", "verde"]</code> <code>In [8]: len(colores)</code> <code>Out[8]: 4</code>
Sublistas	Sintaxis: <code>nombre_lista[inicio : fin+1: salto]</code>
Añadir elementos	Utilizando la función append() : <code>In [70]: numeros = [5, 2, 3, 4, 1]</code> <code>In [71]: numeros.append(8); numeros</code> <code>Out[71]: [5, 2, 3, 4, 1, 8]</code> <code>In [72]: numeros.append([7, 9]); numeros</code> <code>Out[72]: [5, 2, 3, 4, 1, 8, [7, 9]]</code> Concatenando listas con + : <code>In [65]: numeros = [5, 2, 3, 4, 1]</code> <code>In [66]: numeros = numeros + [8]; numeros</code> <code>Out[66]: [5, 2, 3, 4, 1, 8]</code> <code>In [67]: numeros = [6, 7] + numeros; numeros</code> <code>Out[67]: [6, 7, 5, 2, 3, 4, 1, 8]</code>

TIPO DE DATOS LISTA

Operaciones	Descripción
Borrar elementos	Usando el método pop() que borra el elemento ubicado en una posición o el último, por defecto: <pre>In [77]: numeros = [5, 2, 3, 4, 1] In [78]: numeros.pop() Out[78]: 1 In [79]: numeros.pop(0) Out[79]: 5 In [80]: numeros Out[80]: [2, 3, 4]</pre>

Realiza actividad_lista.py

TIPO DE DATOS DICCIONARIO

Un diccionario es un tipo de dato compuesto por pares clave : valor donde clave puede ser cualquier tipo inmutable, habitualmente cadenas de texto o números. En cuanto a los valores, pueden ser de cualquier tipo de datos.

Una característica que los diferencia de las listas es que, en este caso, sus elementos no guardan ningún orden.

TIPO DE DATOS DICCIONARIO

Podemos crear un diccionario de varias formas:

a) Utilizando el operador {}:

```
In [33]: dic = {'nombre': "Mafalda", 'ama': "Los Beatles", 'odia': "sopa"}  
  
In [34]: dic  
Out[34]: {'ama': 'Los Beatles', 'nombre': 'Mafalda', 'odia': 'sopa'}
```

Figura 17: Crear un diccionario usando {}

b) Utilizando el constructor dict() y pasándole los pares clave:valor correspondientes:

```
In [19]: dic = dict(nombre = "Mafalda", ama = "Los Beatles", odia = "sopa")  
  
In [20]: dic  
Out[20]: {'ama': 'Los Beatles', 'nombre': 'Mafalda', 'odia': 'sopa'}
```

Figura 18: Uso de dict() con asignaciones clave=valor

```
In [22]: dic = dict([('nombre', 'Mafalda'), ('ama', 'Los Beatles'), ('odia', 'sopa')])  
  
In [23]: dic  
Out[23]: {'ama': 'Los Beatles', 'nombre': 'Mafalda', 'odia': 'sopa'}
```

Figura 19: Uso de dict() con secuencias de (clave, valor)

c) Creando un diccionario vacío y, después, utilizando el operador de asignación para cada uno de los pares clave:valor que queremos añadir:

```
In [12]: dic = {}  
  
In [13]: dic["nombre"] = "Mafalda"  
  
In [14]: dic["ama"] = "Los Beatles"  
  
In [15]: dic["odia"] = "sopa"  
  
In [16]: dic  
Out[16]: {'ama': 'Los Beatles', 'nombre': 'Mafalda', 'odia': 'sopa'}
```

Figura 20: Creación de un diccionario

TIPO DE DATOS DICCIONARIO

En la siguiente tabla se recogen algunas operaciones con diccionarios:

Operaciones	Descripción
Acceso a elementos	Sintaxis: <i>nombre_diccionario[clave]</i>
Longitud	Utilizaremos la función len() : <code>In [113]: dic1 = {'nombre':'Mafalda', 'ama':"Los Beatles", 'odia':"sopa"}</code> <code>In [114]: len(dic1)</code> <code>Out[114]: 3</code>
Recuperar elementos	Para recuperar la lista de elementos usaremos items() : <code>In [91]: dic1 = {'nombre':"Mafalda", 'ama':"Los Beatles", 'odia':"sopa"}</code> <code>In [92]: dic1.items()</code> <code>Out[92]: dict_items([('ama', 'Los Beatles'), ('nombre', 'Mafalda'), ('odia', 'sopa')])</code> Para recuperar la lista de claves usaremos keys() : <code>In [34]: dic1 = {'nombre':"Mafalda", 'ama':"Los Beatles", 'odia':"sopa"}</code> <code>In [35]: dic1.keys()</code> <code>Out[35]: dict_keys(['ama', 'nombre', 'odia'])</code> Para recuperar la lista de valores usaremos values() : <code>In [1]: dic1 = {'nombre':"Mafalda", 'ama':"Los Beatles", 'odia':"sopa"}</code> <code>In [2]: dic1.values()</code> <code>Out[2]: dict_values(['Los Beatles', 'Mafalda', 'sopa'])</code>

TIPO DE DATOS DICCIONARIO

Operaciones	Descripción
Recorrer diccionario	Utilizando el método keys() para recorrer solamente las <i>claves</i>: <pre>In [82]: dic1 = {'nombre': "Mafalda", 'ama': "Los Beatles", 'odia': "sopa"} In [83]: for k in dic1.keys():: print(k, end=" "): ama nombre odia</pre> Utilizando el método items() para recorrer tanto las <i>claves</i> como los <i>valores</i>: <pre>In [103]: dic1 = {'nombre': "Mafalda", 'ama': "Los Beatles", 'odia': "sopa"} In [104]: for k, v in dic1.items():: print(k, '-->', v): ama --> Los Beatles nombre --> Mafalda odia --> sopa</pre>
Borrar elementos	El método pop() borra el elemento correspondiente a la <i>clave</i> que se pasa como argumento. <u>Sintaxis</u> pop(<i>clave</i> [, <i>valor_defecto</i>])

Realiza actividad_diccionario.py

EXCEPCIONES

Una excepción se produce cuando ocurre un error de ejecución en un programa. En ese momento, el programa acaba y Python muestra un mensaje de error.

Podemos manejar las excepciones que se producen en la ejecución de un programa y dar una respuesta personalizada. Para ello debemos utilizar el bloque try...except.

Su sintaxis es la siguiente:

```
try:  
    instrucciones que pueden provocar una excepción  
except nombre_excepción:  
    instrucciones para el tipo de excepción 1  
except:  
    instrucciones para excepciones generales  
else:  
    instrucciones que se ejecutarán cuando no se produzca una excepción  
finally:  
    instrucciones que se ejecutarán al final del bloque
```

EXCEPCIONES

```
1 try:
2     dato = int(input("Introduce un entero: "))
3     dato = 100 / dato
4 except ValueError:
5     print("Error. El dato no es un entero.")
6 except ZeroDivisionError:
7     print("Error. El dato debe ser distinto de cero.")
8 except:
9     print("\nError genérico.")
10 else:
11     print("El resultado es:", dato)
12 finally:
13     print("Fin del programa.")
```

Figura 21: Especificamos varios tipos de excepciones y el general

Realiza actividad_excepciones.py
Realiza actividad_excepciones2.py
Realiza actividad_excepciones3.py
Realiza actividad_excepciones4.py
Realiza actividad_excepciones5.py

FUENTES DE INFORMACIÓN

- Página oficial del lenguaje Python: <https://www.python.org/>
- Curso CEFIRE CTEM PROGRAMACIÓN AVANZADA CON PYTHON de María Paz Segura Valero
- "Python a fondo" de Óscar Ramírez Jiménez